Project Report

Wazuh, Shuffle and TheHive SOAR Integration

Automating the path from detection to a documented, closed investigation

Author	Mohammed Asim Ahmed Abdullah
Programme	Cyber50 Defense, Blue Team Internship (Task 3)
Date	2 November 2026
Stack	Wazuh SIEM, Shuffle SOAR (self-hosted), TheHive 5.8.1, VMware Workstation

1. Summary

I extended my existing Wazuh and Shuffle SOAR lab by adding TheHive for case management, so that a security event now flows automatically from detection to a tracked, documented investigation. Wazuh detects the event, Shuffle filters and enriches it and creates an alert in TheHive, and I work that alert as a SOC analyst through to a closed case.

I validated the full pipeline with a simulated brute-force attack. From Kali I ran a password-guessing attack over SMB against a Windows endpoint. Wazuh raised **rule 60204, Multiple Windows logon failures (level 10)**; Shuffle forwarded it to TheHive as a **High-severity** alert with the attacker IP, the targeted account and the host as observables; and I triaged it, promoted it to a case, documented containment and closed it as a **true positive with no impact**. The result is a working detect-to-respond SOC pipeline built entirely from open-source tools in my own lab.

2. Objective

My earlier project connected Wazuh to Shuffle, so Wazuh detections were forwarded into a Shuffle workflow. That setup could detect and route an event, but it had nowhere to track the investigation: no record of who looked at an alert, what they found, or whether it was resolved. The objective of this task was to close that gap by adding TheHive, an open-source incident-response platform, and to make the three tools work as one pipeline: Wazuh for detection, Shuffle for automation and enrichment, and TheHive for case management.

A design decision worth stating up front: I did **not** connect Wazuh directly to TheHive, even though a custom integration makes that possible. Wazuh already feeds Shuffle, and Shuffle feeds TheHive, so a direct link would create every alert twice and bypass all of Shuffle's filtering and enrichment. Routing everything through the SOAR keeps a single, clean path (SIEM to SOAR to case management), which is also the standard architecture in a real SOC.

3. Architecture

The lab runs on five VMs in VMware Workstation, all on the same NAT network so they can reach each other. TheHive is the only component added in this task; everything upstream of Shuffle was already in place.

All VMs on the VMware NAT network 192.168.175.0/24

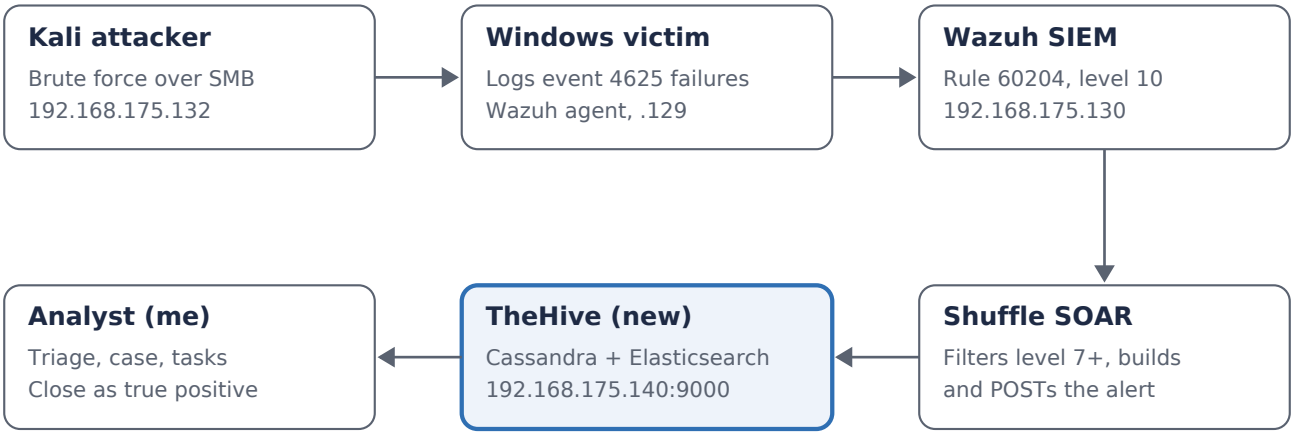


Figure 1: Lab architecture. One attack becomes a tracked TheHive case in four hops.

- **Wazuh (192.168.175.130)** is the detection layer. The agent on the Windows VM ships event logs to the manager, which matches them against its rule set and raises alerts with a level from 0 to 15. Wazuh notices events but has no concept of an investigation.
- **Shuffle SOAR** is the automation layer. It receives Wazuh alerts through a webhook, filters out low-severity noise, calculates a severity, extracts observables and forwards the result to TheHive. It is self-hosted, which matters because the cloud version cannot reach a private lab address.
- **TheHive (192.168.175.140)** is the case-management layer and the new component. It holds alerts, cases, tasks and observables, and keeps an audit trail of who did what, turning a stream of alerts into documented investigations.

TheHive runs three services of its own, all in Docker: TheHive itself, Cassandra (its database) and Elasticsearch (its search index). That is why it has its own VM rather than sharing the Wazuh host.

4. Implementation

4.1 Deploying TheHive

I gave TheHive its own Ubuntu Server 24.04 VM with 8 GB RAM and 4 vCPUs on a static IP (192.168.175.140). It needs three Java services, so sharing the Wazuh host would have exhausted its memory. I deployed it with StrangeBee's official Docker *Testing* profile, which brings up Cassandra, Elasticsearch and TheHive together. Before starting the stack I raised the kernel setting `vm.max_map_count` to 262144, which Elasticsearch requires, and disabled swap, which StrangeBee recommends for Elasticsearch stability. A fresh install runs on a 14-day Platinum trial, which covered the whole build.

```
asim@thehive:~/docker/testing$ docker compose ps
NAME            IMAGE                      COMMAND                  SERVICE         CREATED       STATUS                   PORTS
cassandra       cassandra:4.1.12           "docker-entrypoint.s…"   cassandra       13 days ago   Up 2 minutes (healthy)   7000-7001/tcp, 7199/tcp, 9042/tcp, 9160/tcp
elasticsearch   elasticsearch:8.19.23      "/bin/tini -- /usr/l…"   elasticsearch   13 days ago   Up 2 minutes (healthy)   9200/tcp, 9300/tcp
thehive         strangebee/thehive:5.8.1   "/opt/thehive/entryp…"   thehive         13 days ago   Up 2 minutes (healthy)   0.0.0.0:9000->9000/tcp
asim@thehive:~/docker/testing$
```

Figure 2: All three TheHive services (Cassandra, Elasticsearch, TheHive 5.8.1) running and healthy in Docker.

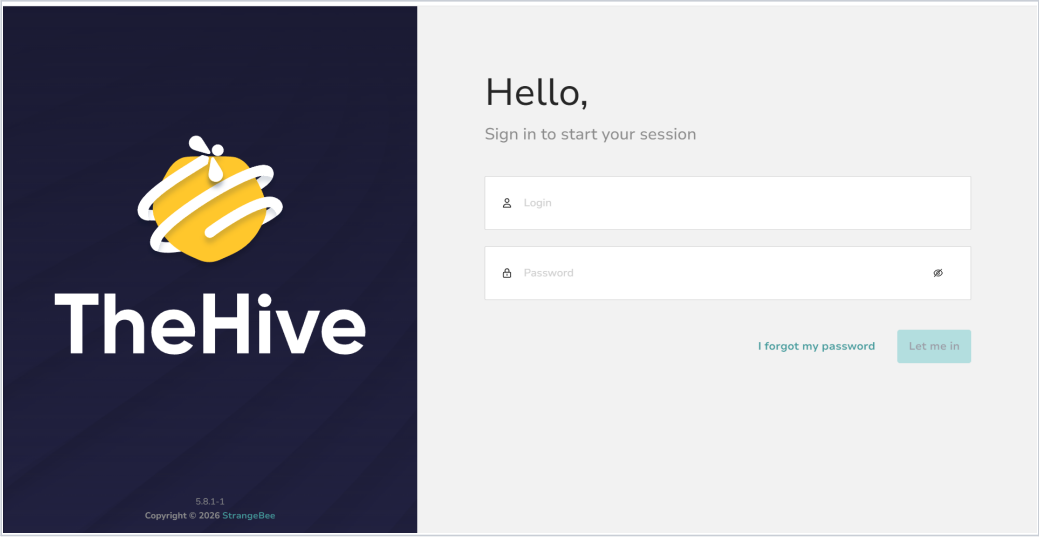


Figure 3: TheHive 5.8.1 web interface reachable at 192.168.175.140:9000/thehive.

4.2 Organisation and least-privilege access

TheHive's default `admin` account lives in a special organisation that cannot hold alerts or cases, so I created a separate organisation, **SOC-Lab**, with two accounts in it:

- `asim@soclab.local`: a normal analyst account, used to work cases.
- `shuffle@soclab.local`: a **service** account with the analyst profile, used only by Shuffle through an API key.

Using a dedicated service account with only the privileges it needs is deliberate: a leaked key does less damage, and TheHive's audit log clearly separates actions taken by the automation from actions I took by hand.

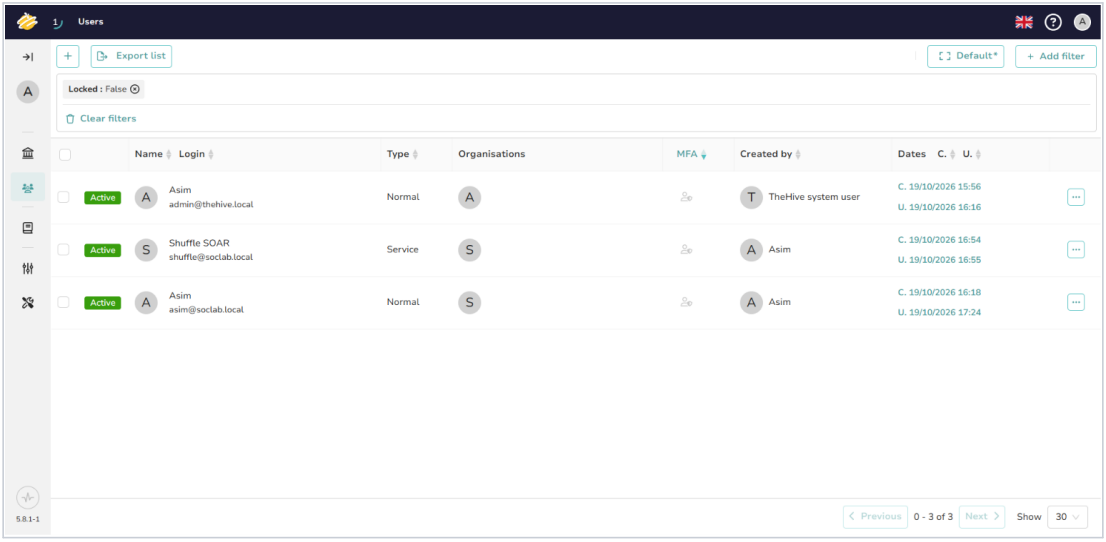


Figure 4: TheHive users: the platform admin, the Shuffle SOAR service account and my analyst account, both in SOC-Lab.

4.3 The Shuffle workflow

I extended my existing Wazuh workflow so that alerts of level 7 or higher are forwarded to TheHive. The workflow runs from a start node that branches to two paths, one to a Discord notification and one to TheHive, so a failure in the Discord step does not stop an alert reaching TheHive.

The TheHive path has two nodes:

1. A **Python node** (`build_thehive_alert`) that takes the raw Wazuh alert and builds a clean TheHive alert. It maps the Wazuh level (0 to 15) to TheHive's severity scale (1 to 4), builds tags including the rule ID and MITRE technique, and extracts observables (source IP, targeted account and host), adding only the fields that exist in that alert.
2. An **HTTP node** (`create_thehive_alert`) that POSTs the Python output to TheHive's alert API, authenticated with the service account's API key.

The severity mapping turns Wazuh level 7 to 9 into Medium, 10 to 12 into High and 13 to 15 into Critical, so the level 10 brute-force rule arrives in TheHive as High. Each alert's source reference is set to the unique Wazuh alert ID, which is also how TheHive de-duplicates: the same event never creates two alerts.

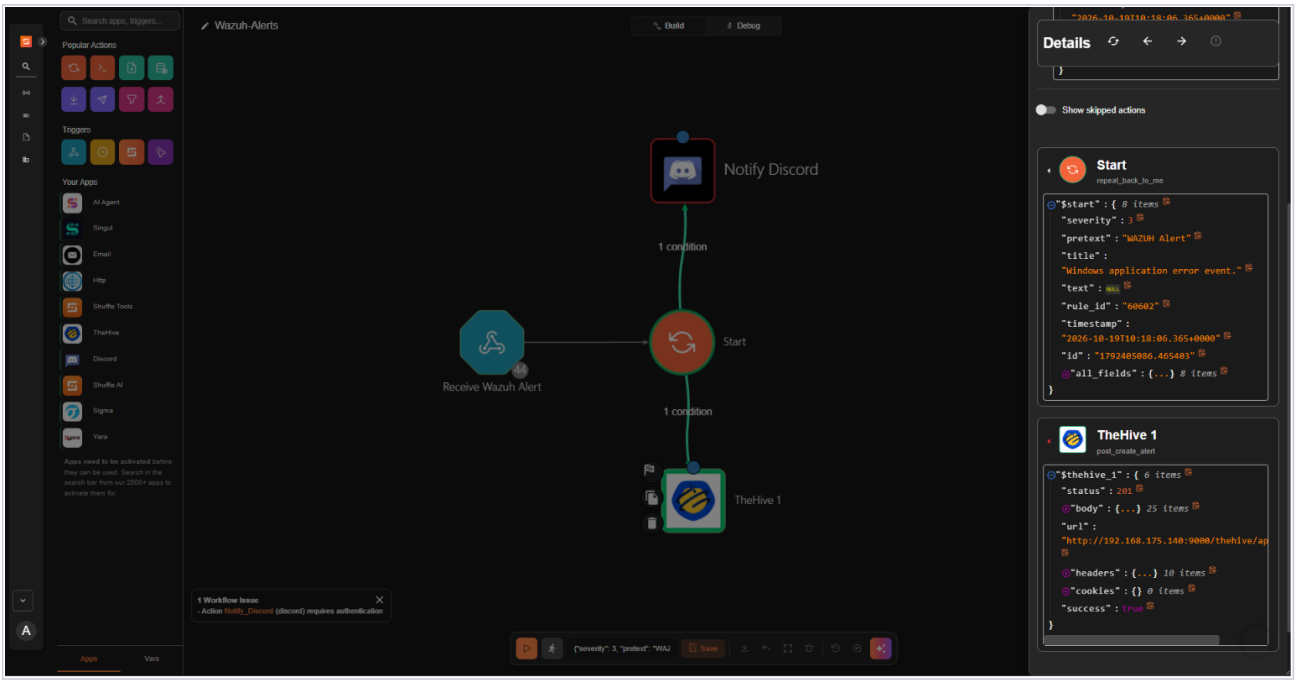


Figure 5: Shuffle workflow: webhook trigger, start node and branches to Discord and TheHive. The TheHive step returns HTTP 201 (alert created).

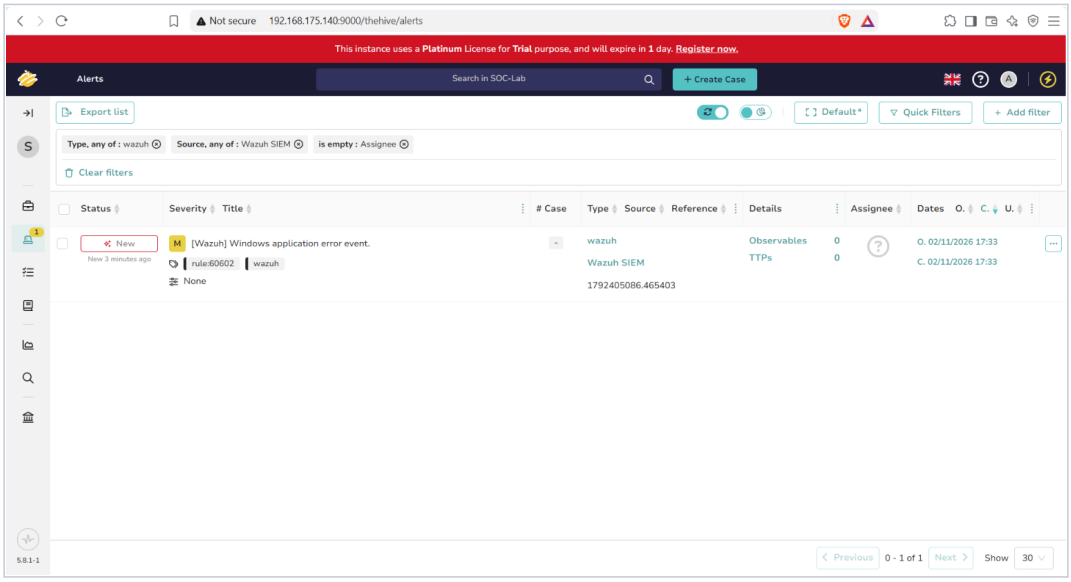


Figure 6: First integration test: a Wazuh alert created in TheHive automatically by Shuffle.

5. Testing the pipeline

To validate the full chain I ran a simulated password-guessing attack. On the Windows victim I confirmed that failed-logon auditing was enabled and that SMB was reachable. Then, from Kali (192.168.175.132), I ran NetExec against the Windows host (192.168.175.129) with a non-existent username and a short password list:

```
nxc smb 192.168.175.129 -u fakeadmin -p /tmp/pw.txt
```

Using a non-existent account means every attempt fails and is logged, without locking out a real account. Each failure produced a Windows Security event 4625. I then traced the attack through every system:

1. **Windows** recorded event 4625 (unknown user name or bad password), collected by the Wazuh agent.
2. **Wazuh** correlated the burst of failures and raised **rule 60204, Multiple Windows logon failures, level 10**, mapped to MITRE ATT&CK T1110 (Brute Force). The dashboard showed 31 authentication failures in the test window.
3. **Shuffle** received the alert, passed it through the level filter and created the alert in TheHive.
4. **TheHive** showed a new High-severity alert, *[Wazuh] Multiple Windows logon failures*, created by the Shuffle service account, with three observables: the host (Windows-Victim), the attacker IP (192.168.175.132) and the targeted account (fakeadmin).

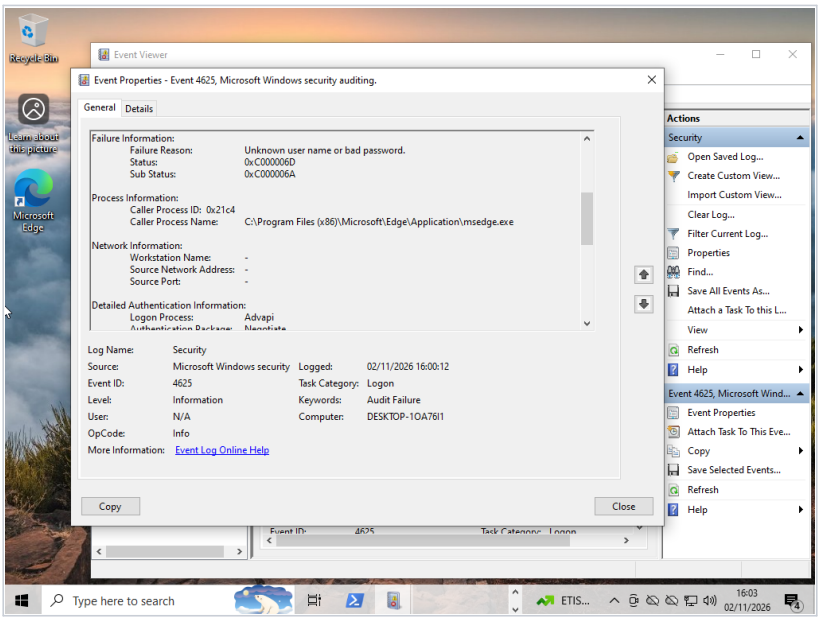


Figure 7: Windows Security event 4625 (failed logon) on the victim, as collected by the Wazuh agent.

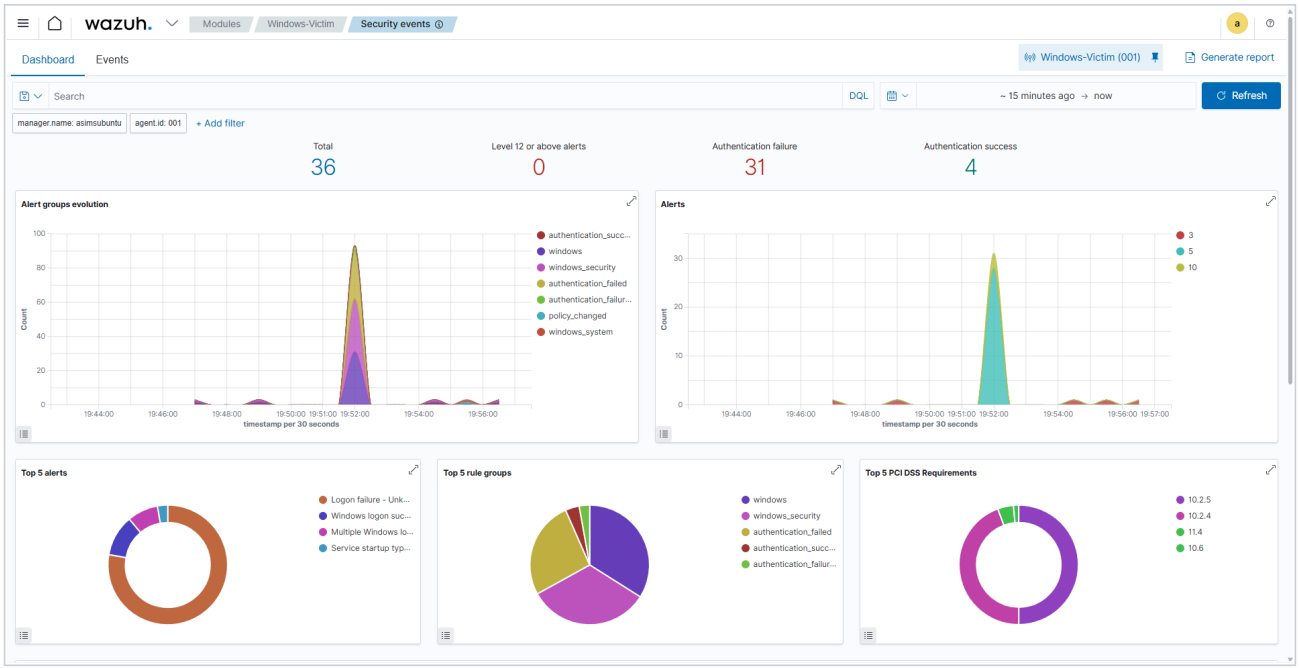
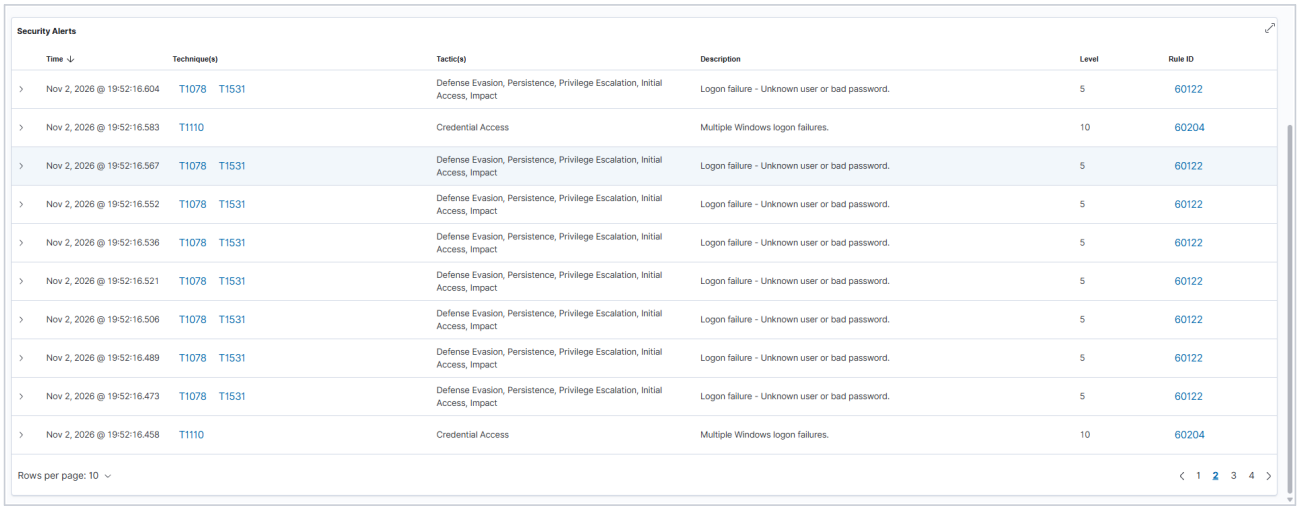


Figure 8: Wazuh dashboard for the Windows agent: 31 authentication failures and a spike of level 10 alerts during the attack.



Security Alerts

Time	Technique(s)	Tactic(s)	Description	Level	Rule ID
Nov 2, 2026 @ 19:52:16.604	T1078 T1531	Defense Evasion, Persistence, Privilege Escalation, Initial Access, Impact	Logon failure - Unknown user or bad password.	5	60122
Nov 2, 2026 @ 19:52:16.583	T1110	Credential Access	Multiple Windows logon failures.	10	60204
Nov 2, 2026 @ 19:52:16.567	T1078 T1531	Defense Evasion, Persistence, Privilege Escalation, Initial Access, Impact	Logon failure - Unknown user or bad password.	5	60122
Nov 2, 2026 @ 19:52:16.552	T1078 T1531	Defense Evasion, Persistence, Privilege Escalation, Initial Access, Impact	Logon failure - Unknown user or bad password.	5	60122
Nov 2, 2026 @ 19:52:16.536	T1078 T1531	Defense Evasion, Persistence, Privilege Escalation, Initial Access, Impact	Logon failure - Unknown user or bad password.	5	60122
Nov 2, 2026 @ 19:52:16.521	T1078 T1531	Defense Evasion, Persistence, Privilege Escalation, Initial Access, Impact	Logon failure - Unknown user or bad password.	5	60122
Nov 2, 2026 @ 19:52:16.506	T1078 T1531	Defense Evasion, Persistence, Privilege Escalation, Initial Access, Impact	Logon failure - Unknown user or bad password.	5	60122
Nov 2, 2026 @ 19:52:16.489	T1078 T1531	Defense Evasion, Persistence, Privilege Escalation, Initial Access, Impact	Logon failure - Unknown user or bad password.	5	60122
Nov 2, 2026 @ 19:52:16.473	T1078 T1531	Defense Evasion, Persistence, Privilege Escalation, Initial Access, Impact	Logon failure - Unknown user or bad password.	5	60122
Nov 2, 2026 @ 19:52:16.458	T1110	Credential Access	Multiple Windows logon failures.	10	60204

Rows per page: 10 1 2 3 4

Figure 9: Wazuh security alerts: individual failures (rule 60122) correlated into rule 60204, level 10, mapped to MITRE T1110.

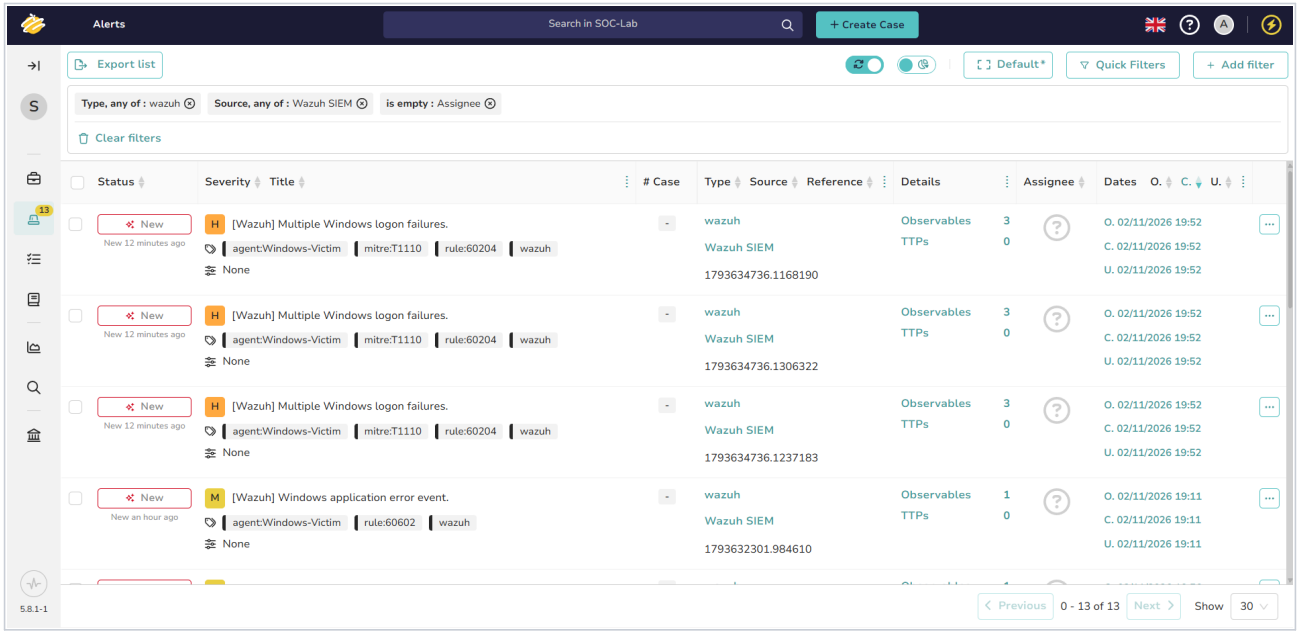


Figure 10: The brute-force alerts in TheHive: High severity, tagged with the agent, MITRE technique and rule ID, each with three observables.

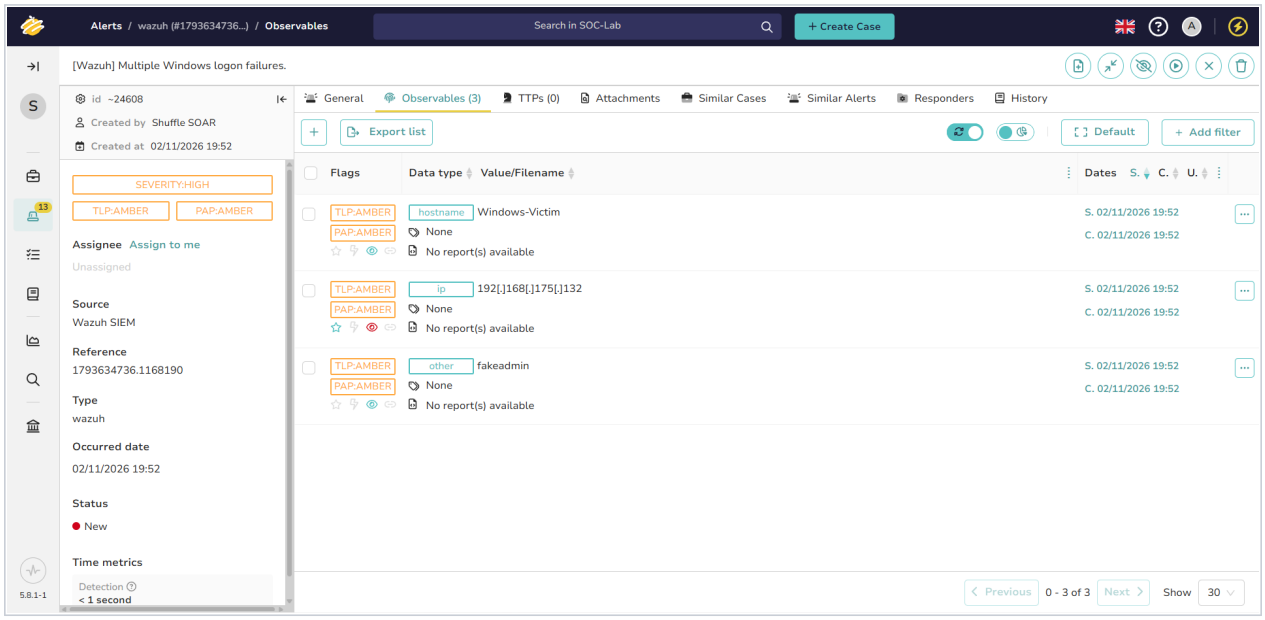


Figure 11: Observables extracted automatically by Shuffle: hostname, attacker IP and targeted account.

Matching the rule ID, level and timestamps across the Wazuh, Shuffle and TheHive views shows the pipeline works end to end for a real event.

6. Incident handling

With the alert in TheHive, I worked it as a SOC analyst would. This is the part that shows the value of adding case management.

1. **Triage.** I opened the alert, reviewed its observables and tags, assigned it to myself and set it to in progress. Many failed logons from one internal IP against a non-existent account is a clear true positive for password guessing (MITRE ATT&CK T1110.001).
2. **Promote to a case.** I created a case from the alert. TheHive copied the observables, tags and description into the case and linked the alert, so it cannot be worked twice.
3. **Tasks.** I added three tasks: *Scope* (check for any successful logon from the source IP), *Contain* (block the source IP) and *Clean up* (document and close). Each task has its own log.
4. **Investigation.** For Scope, I searched Wazuh for successful logons (event 4624) from the attacker IP during the attack window and found none, so no account was compromised. For Contain, I created a Windows Firewall rule blocking the attacker IP (192.168.175.132) and recorded it in the task log.
5. **Enrichment.** I added the MITRE technique to the case and flagged the attacker IP observable as an IOC, tagged ATTACKER.
6. **Closure.** I closed the case as **True Positive** with **no impact** and a summary covering the attack, the detection, the scope check and the containment.

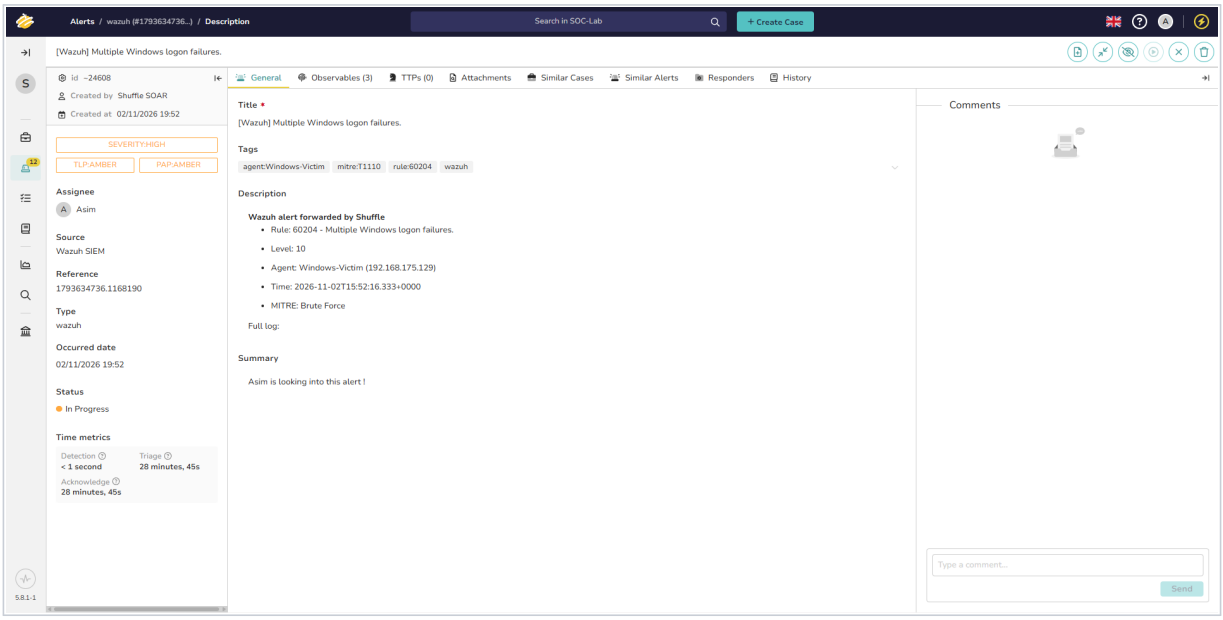


Figure 12: Triage: the alert assigned to me and in progress, with the description built by Shuffle.

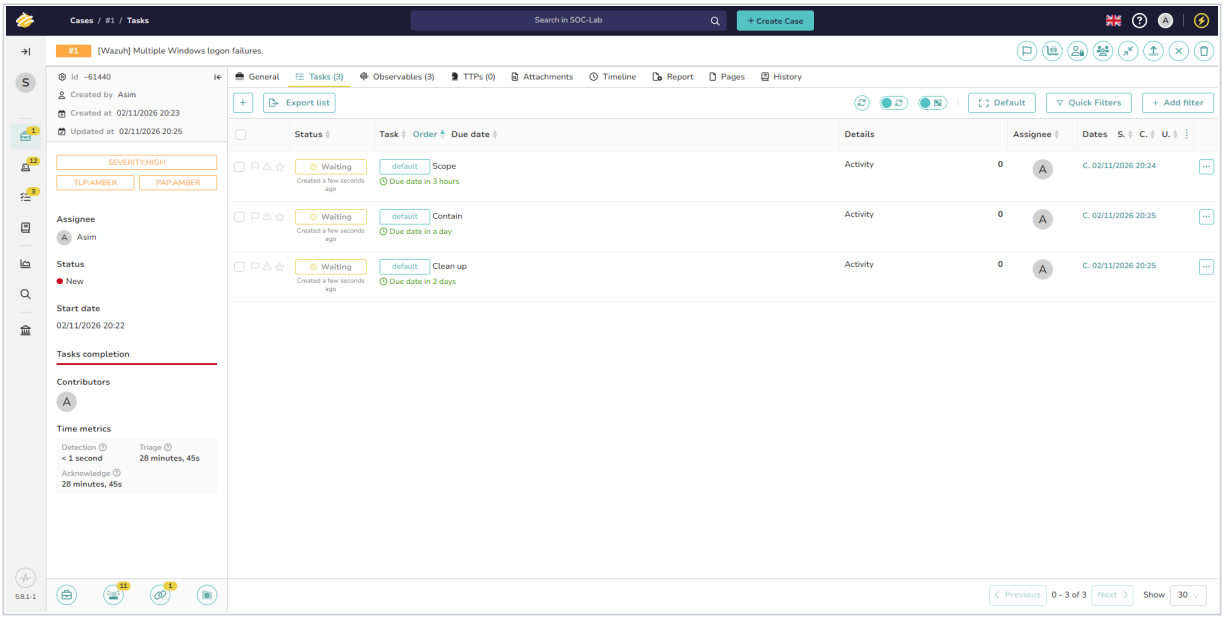


Figure 13: Case #1 with its three tasks: Scope, Contain and Clean up.

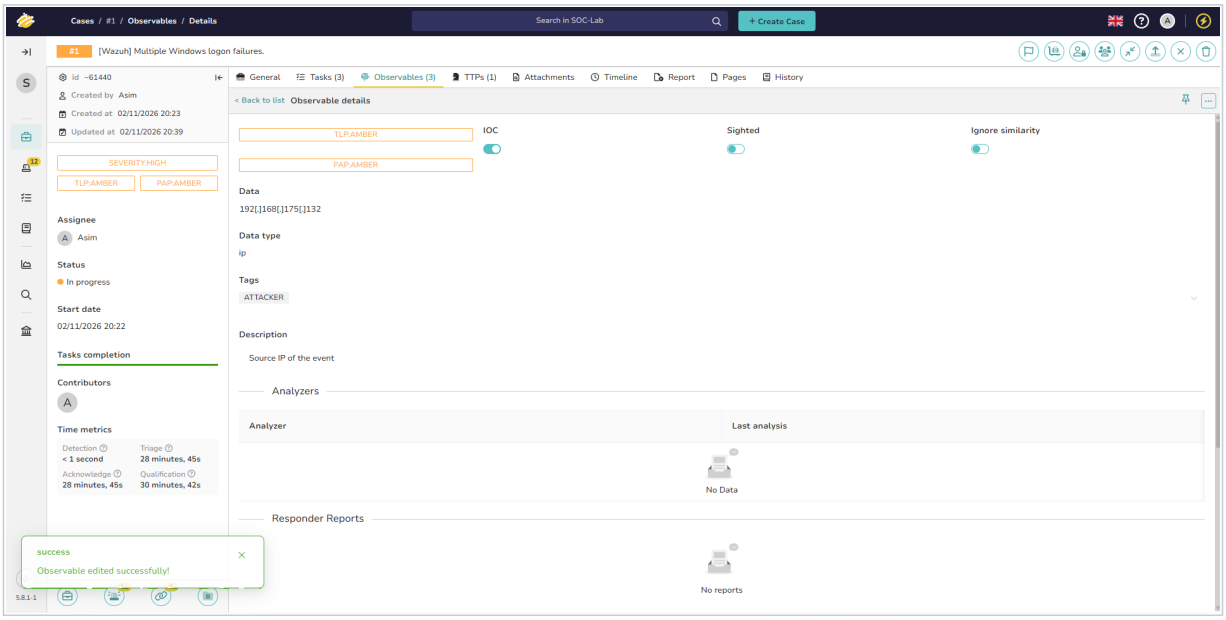


Figure 14: The attacker IP observable flagged as an IOC and tagged ATTACKER.

```
PS C:\Windows\system32> New-NetFirewallRule -DisplayName "Block Kali Attacker" -Direction Inbound -RemoteAddress 192.168
.175.132


Name                            : {8b925319-322b-46c8-8a33-5482f3fe12bb}
DisplayName                     : Block Kali Attacker
Description                     :
DisplayGroup                    :
Group                           :
Enabled                         : True
Profile                         : Any
Platform                        : {}
Direction                       : Inbound
Action                          : Allow
EdgeTraversalPolicy             : Block
LooseSourceMapping              : False
LocalOnlyMapping                : False
Owner                           :
PrimaryStatus                   : OK
Status                          : The rule was parsed successfully from the store. (65536)
EnforcementStatus               : NotApplicable
PolicyStoreSource               : PersistentStore
PolicyStoreSourceType           : Local
RemoteDynamicKeywordAddresses   :
PolicyAppId                     :



PS C:\Windows\system32>
```

Figure 15: Containment: Windows Firewall rule "Block Kali Attacker" created for 192.168.175.132.

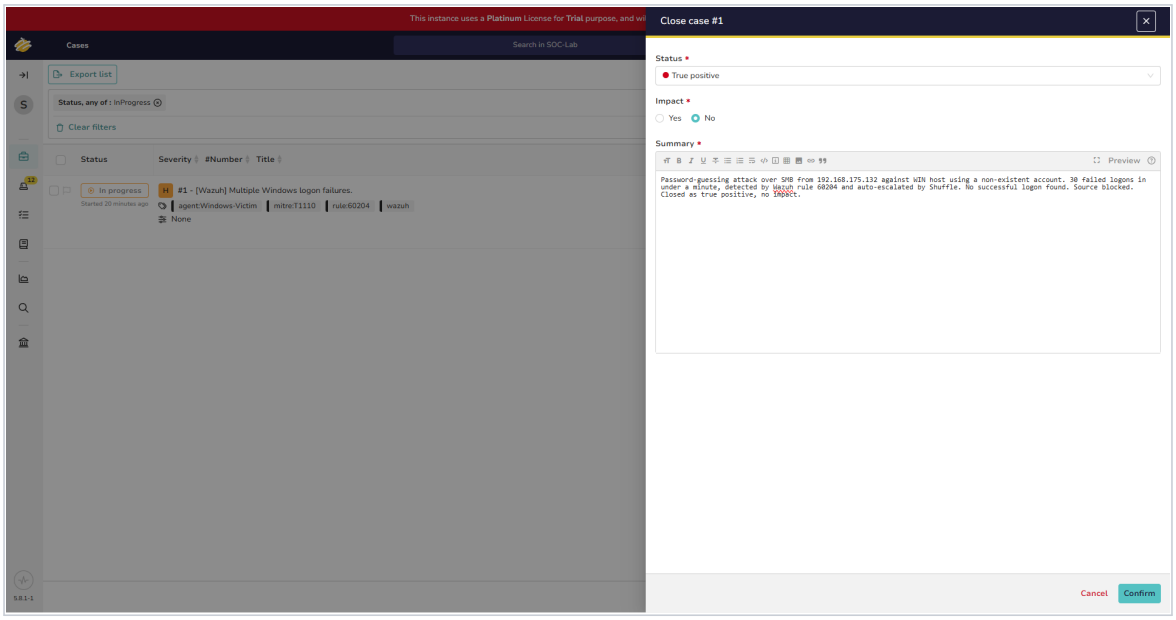


Figure 16: Closing the case as a true positive with no impact, with the incident summary.

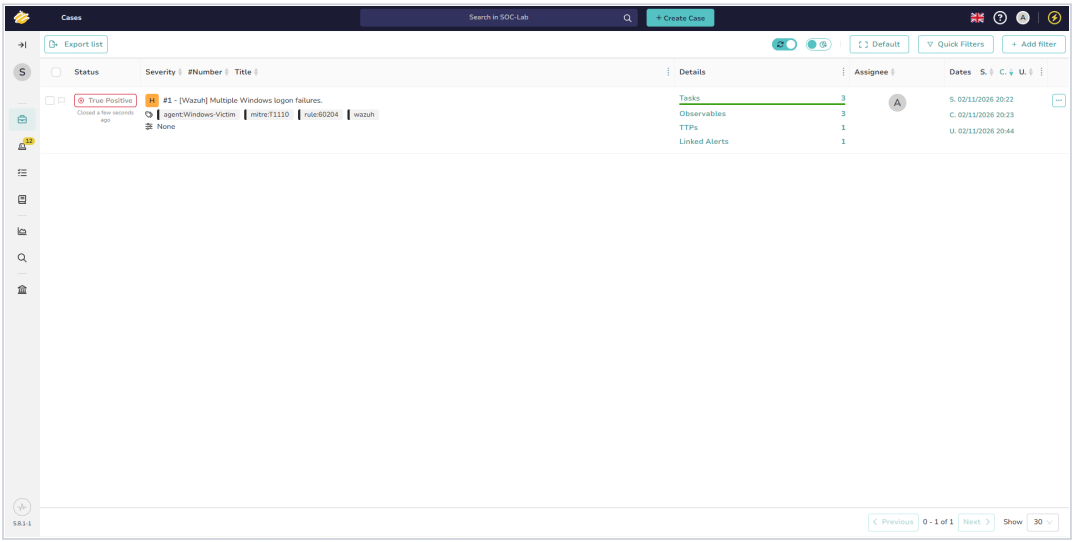


Figure 17: The closed case: True Positive, High severity, 3 tasks completed, 3 observables, 1 TTP, 1 linked alert.

The closed case carries a full audit trail: what happened, who handled it, what was checked and how it was resolved. That record is exactly what Wazuh and Shuffle alone could not provide, and it is the reason for adding TheHive.

7. Problems I hit and how I solved them

These are the issues a real deployment runs into, so they are the most useful part of the project.

Problem	Cause	Fix
StrangeBee licence portal rejected every registration with "Invalid credentials"	A portal-side issue, reproduced with two email addresses and two browsers	Emailed StrangeBee support directly and completed the whole build on the 14-day trial, which needs no account
Shuffle skipped the TheHive node: "not under the startnode"	A Shuffle workflow runs from a single start node; the TheHive node branched off the trigger instead	Added a Shuffle Tools start node that repeats the alert, set it as the start node, and branched Discord and TheHive from it

Problem	Cause	Fix
No alerts appeared in TheHive	Alerts were being viewed from the admin account, whose organisation cannot hold alerts or cases	Used the API key of the shuffle@soclab.local service account in SOC-Lab and viewed alerts as asim@soclab.local
HTTP node body would not be valid JSON	The body wrapped the Python output in an extra pair of braces	Set the body to the raw Python output only ($build_thehive_alert.message)
Early test traffic showed only successful logons (rule 60106)	Those logons were normal activity; they did not represent the attack	Ran NetExec with a non-existent username so every attempt failed, generating 4625 events and rule 60204
No new Shuffle execution for a live alert	Wazuh's webhook in ossec.conf pointed to a different workflow from the one being edited	Compared webhook IDs to find the mismatch; used the execution that actually created the alert as evidence
HTTP 400 "already exists" on reruns	TheHive de-duplicates on the source reference, so replaying the same Wazuh alert is rejected	Expected behaviour; tested with new events instead of reruns

The common thread is that most apparent integration failures were not TheHive faults, but Shuffle workflow logic, authentication scope, or a test that was not generating the right events. Checking which hop an alert stopped at, and reading that tool's own output, was the fastest way to isolate each one.

8. Next steps

- **Automatic response.** Have Shuffle call the Wazuh API to block the attacker IP on the endpoint, with a human approval step before the block runs, turning the lab from detect-and-track into detect, track and respond.
- **Alert grouping.** One attack burst raised rule 60204 several times, creating several TheHive alerts. Grouping them by source IP within a time window would produce one alert per attack.
- **Cortex enrichment.** Add Cortex, TheHive's analysis engine, to run analysers such as reputation lookups on observables directly inside a case.
- **Clean up the live pipeline.** Point Wazuh's webhook at the final workflow and re-select the Discord credential so the full chain runs on live alerts without manual reruns.
- **Licensing.** Activate the free Community licence once StrangeBee validates the account, so the lab keeps working after the trial.
- **Credential hygiene.** The lab uses simple local passwords on an isolated network. Before any real deployment these would be rotated and stored in a secrets manager, and the TheHive API key scoped and rotated regularly.

Appendix: Lab reference

Component	Address	Role
Wazuh SIEM	192.168.175.130	Detection; dashboard on 443, server API on 55000
TheHive	192.168.175.140	Case management; web and API on port 9000 at /thehive
Shuffle SOAR	Self-hosted VM	Automation; webhook on port 3001
Windows victim	192.168.175.129	Monitored endpoint with the Wazuh agent
Kali attacker	192.168.175.132	Attack source

TheHive stack: TheHive 5.8.1 with Cassandra and Elasticsearch, deployed with StrangeBee's Docker Testing profile on Ubuntu Server 24.04.

TheHive accounts (SOC-Lab organisation): `asim@soclab.local` (analyst) for working cases; `shuffle@soclab.local` (service) for the Shuffle API key.

API keys and passwords are not shown in this report.